

Fonctionnalités bonus

Déploiement d'un LLM Open-Source avec Docker Compose et Kubernetes

Introduction

Ce document complète le rapport principal du projet *Déploiement d'un LLM Open-Source avec Docker Compose et Kubernetes*. Il détaille les quatre fonctionnalités optionnelles implémentées au-delà des exigences obligatoires : la restructuration des manifestes avec Kustomize (environnements dev/prod), des NetworkPolicy, des PodDisruptionBudget, et un pipeline de validation continue (CI) sur GitHub Actions.

Par souci de robustesse, aucune de ces fonctionnalités n'a été appliquée directement sur le cluster Minikube du rapport principal, déjà validé et capturé en écran : le risque de saturer à nouveau les ressources de la machine (des incidents de mémoire et de CPU ont déjà été rencontrés et documentés dans le rapport principal) n'en valait pas la peine. Chaque élément a en revanche été rendu et validé localement (`kubectl kustomize + kubeconform`), puis vérifié en conditions réelles via le pipeline CI décrit en section 4.

1 Restructuration Kustomize (environnements dev / prod)

Les neuf manifestes de base ont été déplacés tels quels dans `k8s/base/`, accompagnés d'un fichier `kustomization.yaml` qui les référence. Deux overlays ont ensuite été créés :

- `k8s/overlays/dev/` : espace de noms `llm-devops-dev`, limites CPU/mémoire d'Ollama réduites (2 vCPU / 3 Gi au lieu de 6 vCPU / 5 Gi), hôte d'Ingress `llm-dev.local` — adapté à une itération rapide sur une machine peu puissante.
- `k8s/overlays/prod/` : espace de noms `llm-devops-prod`, stockage Ollama porté à 30 Gi, et ajout des NetworkPolicy et PodDisruptionBudget détaillés ci-dessous.

Chaque overlay ne fait que patcher la base commune : un correctif appliqué dans `base/` profite automatiquement aux deux environnements, sans duplication de YAML. Les deux rendus ont été validés avec `kubeconform`, qui vérifie leur conformité au schéma de l'API Kubernetes :

```
$ kubectl kustomize k8s/overlays/dev | kubeconform -strict -summary
Summary: 9 resources found in 1 file - Valid: 9, Invalid: 0, Errors: 0, Skipped: 0

$ kubectl kustomize k8s/overlays/prod | kubeconform -strict -summary
Summary: 13 resources found in 1 file - Valid: 13, Invalid: 0, Errors: 0, Skipped: 0
```

2 NetworkPolicy

Deux règles réseau, actives uniquement dans l'overlay `prod` :

- Ollama n'accepte du trafic entrant que depuis les pods portant le label `app.kubernetes.io/name: open-webui`, sur le port 11434.
- Open WebUI n'accepte du trafic entrant que depuis le namespace du contrôleur Ingress (`ingress-nginx`), sur le port 8080.

Limite honnête à signaler : une NetworkPolicy n'est effective que si le plugin réseau (CNI) du cluster l'implémente. Le CNI par défaut de Minikube observé ici (`kindnet`, visible dans les pods du namespace `kube-system`) accepte ces objets sans erreur mais ne les fait pas respecter. Ces manifestes sont donc corrects et fonctionneraient tels quels sur un cluster de production utilisant un CNI compatible (Calico, Cilium, Weave), mais leur effet n'a pas pu être vérifié en conditions réelles sur cet environnement de test.

3 PodDisruptionBudget

Un `PodDisruptionBudget` avec `minAvailable: 1` a été défini pour chacun des deux Deployments (overlay prod). Il protège contre les interruptions volontaires (par exemple un `kubectl drain` lors d'une maintenance planifiée d'un nœud) : Kubernetes refusera d'évincer un pod si cela faisait tomber le nombre de répliques disponibles en dessous du seuil défini. Ce mécanisme est complémentaire aux probes, qui gèrent les pannes non planifiées.

4 Pipeline GitHub Actions

Pour valider ce travail dans des conditions représentatives, le contenu de `llm-devops-project` a été extrait dans un dépôt public dédié : github.com/Fa-Syaka-diouf/llm-devops-project. Un pipeline GitHub Actions (`.github/workflows/k8s-validate.yml`) s'y déclenche à chaque modification du dossier `k8s/` : il installe `kubectl`, rend les deux overlays avec Kustomize, les valide avec `kubeconform`, puis vérifie le style des fichiers YAML avec `yamllint`. L'ensemble tourne sur l'infrastructure de GitHub, sans aucune charge sur la machine locale.

Le premier déclenchement du pipeline a d'ailleurs révélé une vraie erreur (fichiers YAML sans retour à la ligne final), corrigée immédiatement — preuve que le pipeline fonctionne réellement et ne se contente pas de toujours réussir. Après correction, le run suivant est passé avec succès :

```
Run: https://github.com/Fa-Syaka-diouf/llm-devops-project/actions/runs/34655853154
Job "validate" : succeeded in 11s
```

```
Summary: 9 resources found in 1 file - Valid: 9, Invalid: 0, Errors: 0
Summary: 13 resources found in 1 file - Valid: 13, Invalid: 0, Errors: 0
yamllint: outcome=success
```